Assignment 1: Network Attack Detection

Part 1: Packet Sniffer Extension

- Implementation Details

  The packet sniffer was built using Python's socket and struct modules to capture and unpack raw network frames at the byte level. THis specifically filters for IPv4 traffic, and extracts the timestamp, source IP, destination IP, etc.

  To handle the high volumes of traffic effectively, the script utilizes a deque to keep a sliding window of 10 minute's worth of packets in the system memory. Additionally, it simulatneously has a dedicated background thread that writes this data to a csv file in 10 second batches.

- Testing Environment

  The sniffer was initialized on the Defense VM using the given command line arguments, and the traffic was generated from the attack VM target the MS-2 target VM.

- Test Results and Screenshots

  - - **Test Case 1: Basic Ping (ICMP Traffic)**

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 packet_sniffer.py -i eth0 -o traffic_log.csv
[sudo] password for kevin:
Starting raw socket packet sniffing on eth0 ...
Logging to traffic_log.csv in batches every 10 seconds.
[2026-03-11 17:00:28] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:28] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:29] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:29] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:30] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:30] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:31] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:31] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:32] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:32] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:33] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:33] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:41] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:41] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:42] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:42] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:43] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:43] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:44] 192.168.10.11 → 192.168.10.13 | Proto: ICMP | Port: - | Flags: ECHO_REQUEST
[2026-03-11 17:00:44] 192.168.10.13 → 192.168.10.11 | Proto: ICMP | Port: - | Flags: ECHO_REPLY
[2026-03-11 17:00:48] 192.168.10.13 → 192.168.10.255 | Proto: UDP | Port: 138 | Flags: -
^Z
zsh: suspended  sudo python3 packet_sniffer.py -i eth0 -o traffic_log.csv
```

```
┌──(kevin㉿kali)-[~]
└─$ ping -c 4 192.168.10.13
PING 192.168.10.13 (192.168.10.13) 56(84) bytes of data.
64 bytes from 192.168.10.13: icmp_seq=1 ttl=64 time=2.26 ms
64 bytes from 192.168.10.13: icmp_seq=2 ttl=64 time=1.68 ms
64 bytes from 192.168.10.13: icmp_seq=3 ttl=64 time=2.28 ms
64 bytes from 192.168.10.13: icmp_seq=4 ttl=64 time=2.31 ms

--- 192.168.10.13 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3011ms
rtt min/avg/max/mdev = 1.680/2.132/2.312/0.261 ms
```

- - **Test Case 2: Port Scan Detection**

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 packet_sniffer.py -i eth0 -o traffic_log.csv
Starting raw socket packet sniffing on eth0...
Logging to traffic_log.csv in batches every 10 seconds.
[2026-03-11 17:01:50] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 22 | Flags: SYN
[2026-03-11 17:01:50] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:01:50] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 443 | Flags: SYN
[2026-03-11 17:01:50] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 49696 | Flags: SYN|ACK
[2026-03-11 17:01:50] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 22 | Flags: RST
[2026-03-11 17:01:50] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 49696 | Flags: SYN|ACK
[2026-03-11 17:01:50] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:01:50] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 49696 | Flags: ACK|RST
^Z
zsh: suspended  sudo python3 packet_sniffer.py -i eth0 -o traffic_log.csv
```

```
┌──(kevin㉿kali)-[~]
└─$ nmap -p 22,80,443 192.168.10.13
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-11 17:29 MDT
Nmap scan report for 192.168.10.13
Host is up (0.0012s latency).

PORT    STATE  SERVICE
22/tcp  open   ssh
80/tcp  open   http
443/tcp closed https
MAC Address: CE:83:CF:61:C8:BA (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 13.21 seconds
```

- - **Test Case 3: SYN Flood Attack Simulation**

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 packet_sniffer.py -i eth0 -o traffic_log.csv
Starting raw socket packet sniffing on eth0 ...
Logging to traffic_log.csv in batches every 10 seconds.
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1086 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1087 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1088 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1089 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1090 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1091 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1092 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1093 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1094 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1095 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: SYN
[2026-03-11 17:02:34] 192.168.10.13 → 192.168.10.11 | Proto: TCP | Port: 1096 | Flags: SYN|ACK
[2026-03-11 17:02:34] 192.168.10.11 → 192.168.10.13 | Proto: TCP | Port: 80 | Flags: RST
```

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 -S -p 80 -i u1000 192.168.10.13
[sudo] password for kevin:
HPING 192.168.10.13 (eth0 192.168.10.13): S set, 40 headers + 0 data bytes
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=0 win=5840 rtt=6.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=1 win=5840 rtt=5.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=2 win=5840 rtt=4.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=3 win=5840 rtt=3.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=4 win=5840 rtt=2.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=5 win=5840 rtt=1.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=6 win=5840 rtt=0.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=7 win=5840 rtt=7.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=8 win=5840 rtt=6.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=9 win=5840 rtt=5.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=10 win=5840 rtt=4.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=11 win=5840 rtt=3.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=12 win=5840 rtt=2.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=13 win=5840 rtt=1.5 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=14 win=5840 rtt=0.5 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=15 win=5840 rtt=7.4 ms
```

- Results and Analysis

  The implmentation proved to be highly efficient, and logs the data every 10 seconds as intended. The sniffer was responsive during the flood, and operated exactly as intended.

Part 2: ICMP Flood Detection

- Implementation Details

The script monitors network traffic at the raw socket level filtering specifically for ICMP packets. It uses a python dictionary to maintain tha real time count of requests that originates from each unique source IP address. In order to identify a malicious flood, the script requires a soruce IP to send more than 50 ICMP requests per second. To filter out temporary netowrk spies, this specified rate must be sustained for atleast 3 seconds. Once triggered, the script executes a historical check using the csv file to verfiy if that same IP address has launched an ICMP flood within the past 30 mins.

- Testing Environment

  The sniffer was initialized on the Defense VM using the given command line arguments, and the traffic was generated from the attack VM target the MS-2 target VM.

- Test Results and Screenshots

  - - **Test Case 1: Normal Ping**

```
┌──(kevin㉿kali)-[~]
└─$ ping -c 5 192.168.10.13
PING 192.168.10.13 (192.168.10.13) 56(84) bytes of data.
64 bytes from 192.168.10.13: icmp_seq=1 ttl=64 time=2.75 ms
64 bytes from 192.168.10.13: icmp_seq=2 ttl=64 time=2.20 ms
64 bytes from 192.168.10.13: icmp_seq=3 ttl=64 time=1.98 ms
64 bytes from 192.168.10.13: icmp_seq=4 ttl=64 time=2.40 ms
64 bytes from 192.168.10.13: icmp_seq=5 ttl=64 time=1.90 ms

--- 192.168.10.13 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4011ms
rtt min/avg/max/mdev = 1.898/2.246/2.752/0.307 ms
```

  - - **Test Case 2: ICMP Flood Attack**

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 -1 --flood -d 120 -a 192.168.10.50 192.168.10.13
HPING 192.168.10.13 (eth0 192.168.10.13): icmp mode set, 28 headers + 120 data bytes
hping in flood mode, no replies will be shown
```

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 icmp_flood_detector.py -i eth0
Starting ICMP Flood Detector on eth0 ...

ALERT: ICMP Flood Detected!
Source: 192.168.10.50 | Rate: >50 requests/sec | Duration: 3 sec
Previous flood detected in last 30 minutes: NO

ALERT: ICMP Flood Detected!
Source: 192.168.10.50 | Rate: >50 requests/sec | Duration: 3 sec
Previous flood detected in last 30 minutes: NO


ALERT: ICMP Flood Detected!
Source: 192.168.10.50 | Rate: >50 requests/sec | Duration: 3 sec
Previous flood detected in last 30 minutes: NO
```

- Results and Analysis

The detector performed as expected, correctly identifying a flood when I triggered it manually. No false positives were recorded, and the CSV history integration sucessfully verified repeated offenders.

-

Part 3: SYN Flood Detection

- Implementation Details

This script inspects raw packets to indefity TCP traffic, and specifically isolates packets where the SYN flag is True and the ACK flag is False. This ensures that the script is strictly counting intial connection requests rather than established traffic. It stores a dictionary that tracks the volume of these SYN packets per source IP address. If an IP address sends more than 100 SYN packets in a single second and maintains the rate for 3 seconds, then a flood alert is triggered. To catch repeated offenders the script is intended to check the csv file to check if that same IP has initiated a similar flood within the last 30 mins.

- Testing Environment

The sniffer was initialized on the Defense VM using the given command line arguments, and the traffic was generated from the attack VM target the MS-2 target VM.

- Test Results and Screenshots

  - **Test Case 1: Normal TCP Connection**

```
┌──(kevin㉿kali)-[~]
└─$ nc -zv 192.168.10.13 80
192.168.10.13: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.10.13] 80 (http) open
```

- - **Test Case 2: SYN Flood Attack**

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 -S -p 80 -i u1000 -a 192.168.10.50 192.168.10.13
HPING 192.168.10.13 (eth0 192.168.10.13): S set, 40 headers + 0 data bytes
```

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 syn_flood_detector.py -i eth0
Starting SYN Flood Detector on eth0 ...

ALERT: SYN Flood Detected!
Source: 192.168.10.50 | Rate: >100 SYN packets/sec | Duration: 3 sec
Previous SYN flood detected in last 30 minutes: NO
```

- - **Test Case 3: Verifying CSV-Based SYN Flood Detection**

```
┌──(kevin㉿kali)-[~]
└─$ sleep 900
```

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 -S -p 80 -i u1000 192.168.10.13
HPING 192.168.10.13 (eth0 192.168.10.13): S set, 40 headers + 0 data bytes
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=0 win=5840 rtt=7.1 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=1 win=5840 rtt=5.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=2 win=5840 rtt=4.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=3 win=5840 rtt=3.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=4 win=5840 rtt=2.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=5 win=5840 rtt=1.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=6 win=5840 rtt=12.4 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=7 win=5840 rtt=11.3 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=8 win=5840 rtt=10.3 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=9 win=5840 rtt=9.2 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=10 win=5840 rtt=8.2 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=11 win=5840 rtt=7.2 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=12 win=5840 rtt=6.1 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=13 win=5840 rtt=5.1 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=14 win=5840 rtt=4.1 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=15 win=5840 rtt=3.0 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=16 win=5840 rtt=2.0 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=17 win=5840 rtt=1.0 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=18 win=5840 rtt=8.0 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=19 win=5840 rtt=6.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=20 win=5840 rtt=5.9 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=21 win=5840 rtt=4.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=22 win=5840 rtt=3.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=23 win=5840 rtt=2.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=24 win=5840 rtt=1.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=25 win=5840 rtt=0.8 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=26 win=5840 rtt=7.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=27 win=5840 rtt=6.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=28 win=5840 rtt=5.7 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=29 win=5840 rtt=4.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=30 win=5840 rtt=3.6 ms
len=44 ip=192.168.10.13 ttl=64 DF id=0 sport=80 flags=SA seq=31 win=5840 rtt=2.6 ms
```

- - **Test Case 4: SYN Flood with IP Spoofing**

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 -S -p 80 -i u1000 --rand-source 192.168.10.13
HPING 192.168.10.13 (eth0 192.168.10.13): S set, 40 headers + 0 data bytes
```

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 syn_flood_detector.py -i eth0
[sudo] password for kevin:
Starting SYN Flood Detector on eth0...
```

- Results and Analysis

The detector was able to initially detect a flood, however, (and this might be because I executed it incorrectly) it was not able to detect the same flood after it was given a sleep command. The script was able to successfully pass all the other test cases.

Part 4: Port Scan Detection

- Implementation Details

THe port scanner tracks the rate of network traffic by monitoring how many unique destination ports a single source IP attempts to contact. It specifically isolates TCP connections by filtering for packets with SYN flags as True and ACK flags as false. The detection mechanism uses three different time tiers. An IP is flagged as a scanner if it targets more than 5 unique ports per second, more than 100 per minute, or more than 100 per 5 minutes. If any of those thresholds are bypassed, then the script cross references the csv file to determine if the IP has a history of scanning within the prior 30 minutes.

- Testing Environment

The sniffer was initialized on the Defense VM using the given command line arguments, and the traffic was generated from the attack VM target the MS-2 target VM.

- Test Results and Screenshots

- - **Test Case 1: Normal Service Connection**

```
┌──(kevin㉿kali)-[~]
└─$ nc -zv 192.168.10.13 22
192.168.10.13: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.10.13] 22 (ssh) open
```

- - **Test Case 2: Port Scanning with Your Own Scanner**

```
┌──(kevin㉿kali)-[~/Documents/Lab-2]
└─$ python3 tcp_scanner.py 192.168.10.13
Scanning TCP ports on 192.168.10.13 ...
[*] Port 21/tcp is open
[*] Port 22/tcp is open
[*] Port 23/tcp is open
[*] Port 25/tcp is open
[*] Port 53/tcp is open
[*] Port 80/tcp is open
[*] Port 111/tcp is open
[*] Port 139/tcp is open
[*] Port 445/tcp is open
[*] Port 512/tcp is open
[*] Port 513/tcp is open
[*] Port 514/tcp is open
```

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 port_scanner_detector.py -i eth0
Starting Port Scanner Detector on eth0 ...

ALERT: Port Scanner Detected!
Source: 192.168.10.11 | Fan-Out Rate: 6 | Reason: Exceeded 5 ports/sec threshold
Previous port scan detected in last 30 minutes: NO

ALERT: Port Scanner Detected!
Source: 192.168.10.11 | Fan-Out Rate: 6 | Reason: Exceeded 5 ports/sec threshold
Previous port scan detected in last 30 minutes: NO
```

- - **Test Case 3: Port Scanning with Nmap**

```
┌──(kevin㉿kali)-[~/Documents/Lab-2]
└─$ nmap -p 1-1024 192.168.10.13
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-11 18:13 MDT
Nmap scan report for 192.168.10.13
Host is up (0.0021s latency).
Not shown: 1012 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
MAC Address: CE:83:CF:61:C8:BA (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 14.30 seconds
```

```
┌──(kevin㉿kali)-[~/Documents/Assignment_1]
└─$ sudo python3 port_scanner_detector.py -i eth0
Starting Port Scanner Detector on eth0 ...
ALERT: Port Scanner Detected!
Source: 192.168.10.11 | Fan-Out Rate: 6 | Reason: Exceeded 5 ports/sec threshold
Previous port scan detected in last 30 minutes: NO
ALERT: Port Scanner Detected!
Source: 192.168.10.11 | Fan-Out Rate: 6 | Reason: Exceeded 5 ports/sec threshold
Previous port scan detected in last 30 minutes: NO
ALERT: Port Scanner Detected!
Source: 192.168.10.11 | Fan-Out Rate: 6 | Reason: Exceeded 5 ports/sec threshold
Previous port scan detected in last 30 minutes: NO
```

- Results and Analysis

The detector successfully idenfties the my own scanner from Lab 2, as well as the nmap scan.

-

Findings and Recommendations

- Challenges Faced

  The biggest challenge I faced was writing the code, it was very high level and I had to reference the past Labs as well as get help from some AI and online sources to help me write the code. The high level logic of how each script should work is very difficult to think through. The execution of each part is fairly simple afterwards.

- Suggested Improvements

- **Reducing False Positives:** One way to reduce false positives would be to track the SYN floods by specifically targeting incomplete handhsakes and only triggering an alert if a high volume of SYN pakcets is received but the corresponding ACK packets from the client are never completed.
  - - **Optimizing Performance:** Instead of using a dictionary, I could use a set which would drastically reduce lookup times and improve the efficiency of the script.
  - - **Enhancing Real-World Deployment:** Instead of simply priting a message that alerts the terminal of a flood or malicious attack, it could be enhanced by automatically dropping packets and actively blocking the offenders. Furthermore, it could alert an employee and bring it to their attention.
  -